Langage C avancé

Laurent Bellon

2025-2026

Préparation de l’Environnement de Développement C

Outils essentiels pour coder, compiler et déboguer

L’environnement de développement est un ensemble d’outils indispensables pour écrire, compiler, déboguer et exécuter des programmes en langage C.

Cette présentation explore :

- Le compilateur **GCC**
- L’éditeur de texte **VS Code**
- L’environnement de développement intégré **Code::Blocks**

GCC : Le Compilateur Standard du C

GCC (GNU Compiler Collection) est un compilateur libre, open-source et standard de facto pour le langage C, développé par la Free Software Foundation.

Caractéristiques :

- **Compatibilité étendue :** Linux, Windows, macOS, architectures variées.
- **Performance :** Génère du code machine optimisé.
- **Diagnostics :** Options puissantes pour avertissements et débogage.
- **Intégration :** Facilité d'intégration avec éditeurs et IDE.

Installation rapide :

- **Linux :** `sudo apt install gcc` (souvent préinstallé)
- **Windows :** Inclus dans MinGW ou WSL
- **macOS :** `xcode-select --install` (via Xcode Command Line Tools)

Compilation et exécution de base :

```
# Exemple : un fichier `hello.c`
gcc hello.c -o hello
./hello
```

Le compilateur transforme votre code source en un programme exécutable.

Visual Studio Code : L'Éditeur Léger et Extensible

VS Code est un éditeur de texte léger, gratuit et open-source, très populaire et extensible.

Fonctionnalités clés pour le C :

- **Support multi-langages :** via des extensions, notamment l'extension officielle "C/C++" par Microsoft.
- **Intégration complète :** Débogueur, gestionnaire de tâches (pour la compilation), terminal intégré.
- **Productivité :** Intellisense (autocomplétion et suggestions de code).
- **Interface :** Moderne et hautement personnalisable.

Compilation intégrée avec `tasks.json` : VS Code permet d'automatiser la compilation en un clic, par exemple avec GCC.

```
{
    "version": "2.0.0",
    "tasks": [{
        "type": "shell",
        "label": "compiler C",
        "command": "gcc",
        "args": ["-g", "${file}", "-o", "${fileBasenameNoExtension}"],
        "group": "build"
    }]
}
```

Ce fichier configure VS Code pour appeler GCC automatiquement.

Code::Blocks : L’IDE Complet pour C/C++

Code::Blocks est un Environnement de Développement Intégré (IDE) open-source, conçu spécifiquement pour le développement en C/C++.

Caractéristiques principales :

- **Interface complète :** Regroupe gestionnaire de projets, éditeur de code et compilateur intégré.
- **Débogueur graphique :** Facilite la recherche et la correction d’erreurs.
- **Multi-plateforme :** Disponible sur Windows, Linux et macOS.
- **Configuration simplifiée :** Intégration transparente avec GCC.

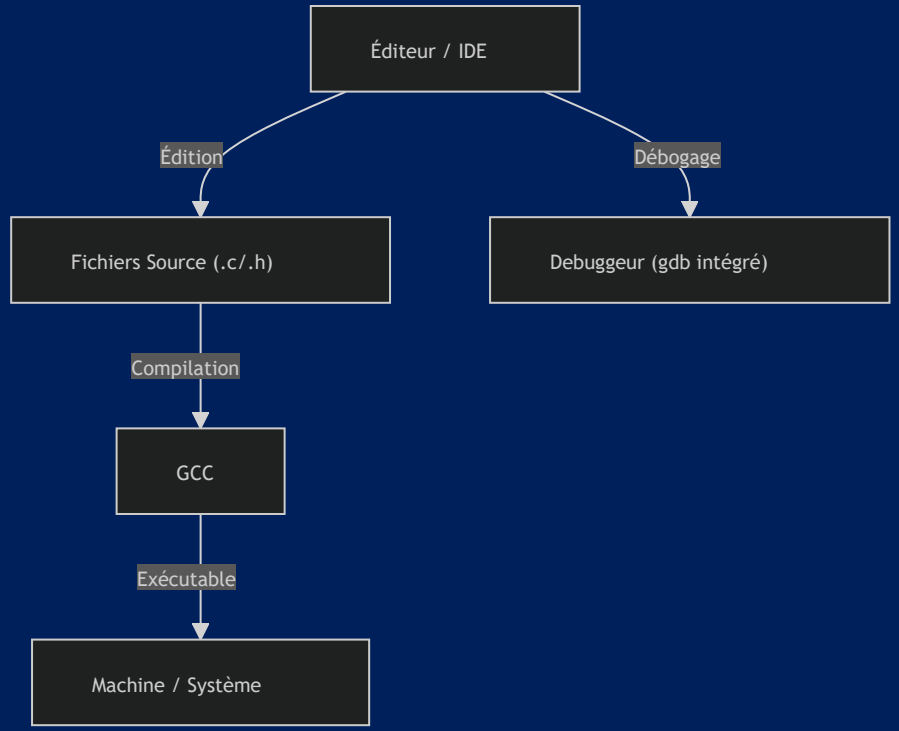
Utilisation :

- Création de projets C/C++ simplifiée.
- Compilation et débogage via des menus graphiques intuitifs.

Idéal pour un développement structuré avec tous les outils à portée de main.

Comprendre l’Écosystème des Outils C

Voici comment les différents outils interagissent au sein de votre environnement de développement :



Résumé des outils :

Outil	Type	Points forts	Usage typique
GCC	Compilateur	Performance, portabilité	Compilation rapide en ligne de commande
VS Code	Éditeur léger	Très personnalisable, extensions	Programmeurs préférant flexibilité
Code::Blocks	IDE complet	Intégration, facilité d'utilisation	Développement structuré projets C

Conclusion & Ressources

Cette présentation fixe les fondations matérielles pour coder efficacement en C à travers des outils modernes polyvalents et accessibles.

Sources utilisées :

- GNU GCC official website - https://gcc.gnu.org/
- Visual Studio Code - https://code.visualstudio.com/docs/languages/cpp
- Code::Blocks - http://www.codeblocks.org/
- TutsMake - How to compile C program using GCC
- Microsoft Docs - VS Code C++ Tutorial

Préparation de l’Environnement de Développement C

Introduction à l’Environnement C

Pourquoi est-ce crucial ?

- L’installation d’un compilateur et d’un IDE est une étape incontournable pour programmer en C.
- Cela permet de transformer votre code source en programme exécutable et de gérer vos projets.

Nous aborderons :

1. L’installation de GCC (le compilateur C incontournable).
2. L’installation de Code::Blocks (un IDE populaire et adapté aux débutants).

Installation de GCC (GNU Compiler Collection)

GCC : Le Compilateur C Incontournable

- Transforme le code source C en exécutable.

1. Sur Windows (via MinGW-w64)

- **Téléchargement :** Site officiel MinGW-w64 (https://www.mingw-w64.org/) ou installateur winlibs.com.
- **Installation :** Choisissez la version (64/32 bits) et l'architecture (x86_64), spécifiez un dossier (ex: `C:\mingw-w64`).
- **PATH :** Ajoutez le chemin du dossier `bin` de MinGW (ex: `C:\mingw-w64\bin`) à la variable d'environnement PATH.
- **Vérification :** Ouvrez `cmd` et tapez `gcc --version`.

2. Sur Linux

- Ouvrez un terminal et lancez :

```
sudo apt update
sudo apt install build-essential
```

- `build-essential` installe GCC et les outils de compilation.
- **Vérification :** `gcc --version`.

3. Sur macOS

- Installez les outils en ligne de commande Xcode :

```
xcode-select --install
```

- Ou via Homebrew :

```
brew install gcc
```

Installation de Code::Blocks (IDE)

Code::Blocks : L'Environnement de Développement Intégré (IDE)

- Open-source, facilite la gestion de projets C/C++.
- Intègre facilement GCC.

1. Sur Windows

- **Téléchargement :** Site officiel Code::Blocks (http://www.codeblocks.org/downloads).
- **Recommandé :** Choisissez le package complet avec MinGW intégré (ex: `codeblocks-20.03mingw-setup.exe`).
- **Installation :** Exécutez le fichier, suivez l'assistant (options par défaut).

2. Sur Linux

- Installer via gestionnaire de paquets :

```
sudo apt install codeblocks
```

- Code::Blocks détectera automatiquement GCC s'il est déjà installé.

3. Sur macOS

- **Téléchargement :** Fichier `.dmg` officiel de Code::Blocks (http://www.codeblocks.org/downloads).
- Suivez les instructions d'installation.

Premier Lancement et Test

Après l'installation de Code::Blocks (Windows)

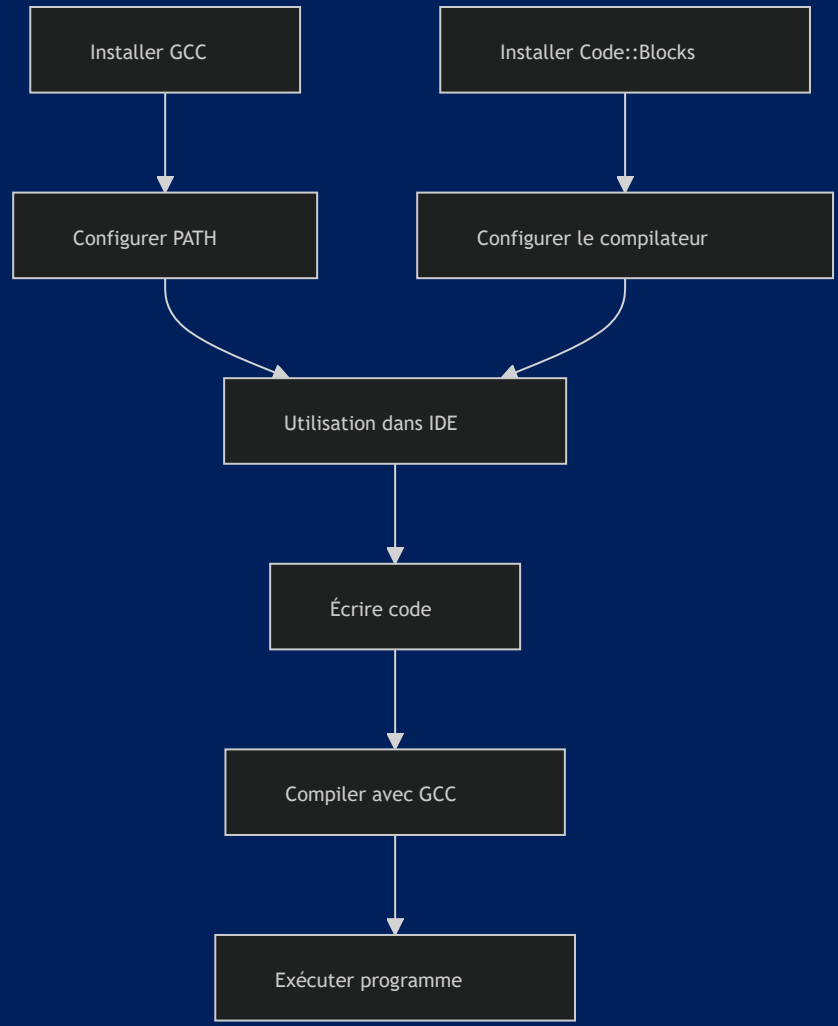
1. **Lancement et configuration :**
    1. Lancez Code::Blocks.
    2. Dans `Settings > Compiler`, vérifiez que le compilateur GNU GCC est sélectionné.
2. **Création d'un projet simple :**
    1. `File > New > Project > Console Application > C`.

3. **Compilation et Exécution :**
    1. L'IDE propose deux boutons principaux :
        1. **Build** (ou "Compiler") : transforme le code source en exécutable.
        2. **Run** : exécute le programme généré.

Exemple de code à tester (dans `main.c`) :

```c
#include <stdio.h>

int main() {
    printf("Hello, Code::Blocks and GCC!\n");
    return 0;
}
```


Installer GCC
Installer Code::Blocks
Configurer PATH
Configurer le compilateur
Utilisation dans IDE
Écrire code
Compiler avec GCC
Exécuter programme

Description du flux :

1. **Installation de GCC** : Le compilateur est mis en place.
2. **Configuration du PATH** : Permet au système de trouver GCC.
3. **Installation de Code::Blocks** : L'IDE est installé.
4. **Configuration du compilateur** : L'IDE est connecté à GCC.
5. **Utilisation dans l'IDE** : L'environnement est prêt.
6. **Écriture du code** : Le développeur rédige son programme.
7. **Compilation avec GCC** : L'IDE utilise GCC pour transformer le code en exécutable.
8. **Exécution du programme** : Le programme est lancé.

Conclusion & Ressources

Ce qu'il faut retenir :

- Vous disposez maintenant d'un environnement stable et complet.
- Cet environnement est prêt pour :
  - Écrire du code C.
  - Compiler vos programmes.
  - Exécuter vos applications.
  - Déboguer vos projets.

Sources utilisées :

- GCC official site - https://gcc.gnu.org/install/
- MinGW-w64 - https://www.mingw-w64.org/
- Code::Blocks download - http://www.codeblocks.org/downloads
- Ubuntu Documentation - https://help.ubuntu.com/community/InstallingCompilers
- Homebrew for macOS - https://brew.sh/

Rappels et Approfondissements : Types de données et Opérateurs avancés en C

Le langage C, par sa flexibilité et sa puissance, est fondamental pour le développement de programmes efficaces et performants. Une maîtrise approfondie de ses types de données et opérateurs est la clé pour écrire du code sûr et optimisé.

Les Types de Données : Les Fondations

Comprendre comment C gère les données est crucial.

Types de base :

- `int` : Entier (généralement 4 octets)
- `float` : Nombre à virgule flottante simple précision
- `double` : Nombre à virgule flottante double précision
- `char` : Caractère (1 octet, code ASCII)

Types dérivés et avancés (partie 1) :

- **Tableaux (Arrays)**
  - Séquence contiguë d'éléments de même type.

```c
int tab[5] = {1, 2, 3, 4, 5};
```

- **Pointeurs**
  - Variable stockant l'adresse mémoire d'une autre variable. Cœur de la gestion mémoire en C.

```c
int a = 10;
int *ptr = &a; // ptr pointe vers a
```

- **Structures (struct)**
  - Regroupement d'éléments hétérogènes sous un même type.

```c
struct Point { int x; int y; };
struct Point p1 = {10, 20};
```

Types dérivés et avancés (partie 2) :

- **Unions**
  - Stockent différentes données au même emplacement mémoire (mutuellement exclusives).
- **Enums (énumérations)**
  - Types symboliques représentant des constantes entières nommées.

```
enum Days {LUNDI, MARDI, MERCREDI};
enum Days today = MARDI;
```

```c
#include <stdio.h>

typedef enum {
    DEVICE_OFF,
    DEVICE_ON,
    DEVICE_ERROR
} DeviceStatus;

int main() {
    DeviceStatus status = DEVICE_OFF;

    if (status == DEVICE_OFF) {
        printf("L'appareil est éteint.\n");
    }

    status = DEVICE_ON;

    if (status == DEVICE_ON) {
        printf("L'appareil est allumé.\n");
    }

    return 0;
}
```

```c
#include <stdio.h>

typedef enum {
    TYPE_INT,
    TYPE_FLOAT,
    TYPE_STRING
} ValueType;

typedef union {
    int i;
    float f;
    const char *s;
} Value;

typedef struct {
    ValueType type;
    Value value;
} TypedValue;
```

-->Suite

```c
int main() {
    TypedValue tv;

    tv.type = TYPE_INT;
    tv.value.i = 42;

    if (tv.type == TYPE_INT) {
        printf("Valeur entière : %d\n", tv.value.i);
    }

    tv.type = TYPE_STRING;
    tv.value.s = "Hello world";

    if (tv.type == TYPE_STRING) {
        printf("Valeur chaîne : %s\n", tv.value.s);
    }

    return 0;
}
```

Les Opérateurs Avancés : Manipulation Fine

Opérateurs d'adresse et de pointeur :

- `&` (adresse de) : Récupère l'adresse d'une variable.
- `*` (déréférencement) : Accède à la valeur pointée par un pointeur.

```
int a = 5;
int *p = &a;     // p stocke l'adresse de a
printf("%d\n", *p); // affiche 5
```

Opérateurs bit à bit : Manipulent les bits d'une valeur entière, essentiels en programmation bas niveau.

- `&` (ET bit à bit)
- `|` (OU bit à bit)
- `^` (OU exclusif bit à bit)
- `~` (complément à un bit à bit)
- `<<` (décalage à gauche)
- `>>` (décalage à droite)

```
unsigned char a = 5;       // binaire: 00000101
unsigned char b = a << 1;  // décalage à gauche → 00001010 = 10
```

Les Opérateurs Avancés : Logique et Contrôle

Opérateur ternaire : Forme compacte d'un `if-else` qui retourne une valeur.

```
int y = (x > 0) ? 1 : -1;
```

Opérateurs d'incrémentation et décrémentation (`++` , `--`) : Augmentent ou diminuent la valeur d'une variable de 1. Leur position (préfixe `++i` vs postfixe `i++`) impacte l'ordre d'évaluation.

Opérateurs de conversion de type (cast) : Forcent la conversion d'un type en un autre.

```
float f = 3.14;
int i = (int)f; // i vaut 3
```

Exemple global illustratif :

```c
#include <stdio.h>
struct Coord { int x; int y; };
int main() {
    int a = 10; int *p = &a;
    printf("Adr de a : %p\n", (void*)p); // Pointeur
    printf("Val pointée : %d\n", *p); // Déréférencement

    struct Coord pt = {5, 7}; struct Coord *ppt = &pt;
    printf("Coords : (%d, %d)\n", ppt→x, ppt→y); // Structure & Pointeur

    unsigned char val = 1 << 3; // Décalage bit à bit
    printf("1 << 3 = %d\n", val);

    int b = -10; int signe = (b ⩾ 0) ? 1 : -1; // Ternaire
    printf("Signe de b : %d\n", signe);
    return 0;
}
```

Ce qu’il faut retenir et nos sources

Ce qu’il faut retenir : Ce cours fournit un cadre solide sur les types de données et opérateurs avancés en C. Ces bases sont indispensables pour aborder des sujets plus complexes comme la gestion mémoire, les algorithmes performants, et les structures de données personnalisées, ouvrant la voie à des développements plus sophistiqués et robustes.

Sources utilisées :

- Descriptif Programmation C 2024-2025 - Scribd
- Langage C avancé - ENSTA (PDF)
- Comprendre les types et opérateurs en C - Blog Alphorm
- C documentation et références traditionnelles (ISO C standards)

Rappels et Approfondissements: Portée des variables (Storage Classes) en langage C

En langage C, la portée, la durée de vie (lifetime) et la visibilité des variables sont régies par les classes de stockage (« storage classes »). Ces classes définissent où la variable est accessible, combien de temps elle existe en mémoire, et si elle est visible en dehors d'un fichier source. Comprendre ces concepts permet d'écrire un code clair, efficace et évite des erreurs liées à la gestion des variables.

Définitions clés

- **Portée (Scope)** : Région du programme où une variable est accessible.
- **Durée de vie (Lifetime)** : Temps pendant lequel une variable conserve une valeur en mémoire.
- **Lien (Linkage)** : Visibilité d'un symbole (variable/fonction) entre plusieurs fichiers sources.

L'ensemble de ces caractéristiques sont définies via les classes de stockage suivantes.

`auto`

- Par défaut, toutes les variables locales.
- Durée : vie du bloc dans lequel elles sont définies (ex: une fonction).
- Pas de visibilité hors du bloc.

Exemple `auto` :

```c
void fonction() {
    int val = 5;  // auto implicite
    printf("%d\n", val);
}
```

`register`

- Indique au compilateur de suggérer un stockage dans un registre CPU (accès rapide).
- Ne peut pas avoir l'adresse prise via `&` (généralement).
- Durée de vie similaire à `auto`.

Exemple `register` :

```c
void boucle() {
    register int i;
    for(i = 0; i < 10; i++) {
        printf("%d ", i);
    }
}
```

`static`

- **Variable locale :** Conserve sa valeur entre plusieurs appels à la fonction. Initialisée une seule fois.
- **Variable globale :** Limite sa visibilité au fichier source (linkage interne).
- **Durée de vie :** celle de tout le programme.

Exemple `static` local :

```c
void compteur() {
    static int count = 0;  // initialisée une seule fois
    count++;
    printf("Compteur = %d\n", count);
}
```

Chaque appel à `compteur` affichera une valeur incrémentée.

`extern`

- Déclare une variable ou fonction définie dans un autre fichier source.
- Elle n’est pas allouée dans le fichier où elle est déclarée.
- Le linkage est externe.

Exemple `extern` :

Dans fichier1.c :

```c
int valeur = 42;
```

Dans fichier2.c :

```c
extern int valeur;
void afficher() {
    printf("%d\n", valeur);
}
```

Les Classes de Stockage en C

Classe de stockage	Portée	Durée de vie	Linkage	Description
auto	Locale (bloc)	Durée automatique	Aucun	Variable locale classique, créée à l'entrée du bloc, détruite à la sortie. Par défaut pour les variables locales.
register	Locale (bloc)	Durée automatique	Aucun	Variable locale suggérée pour stockage dans registre CPU, accès rapide. Variable locale avec possibilité d'optimisation.
static	Locale ou globale	Persistante (durée du programme)	Aucun (locale) ou Interne (globale)	Local statique : variable locale qui conserve sa valeur entre les appels. Globale statique : variable ou fonction visible seulement dans le fichier source (linkage interne).
extern	Globale	Persistante (durée du programme)	Externe	Variable/fonction définie dans un autre fichier ou dans le même fichier, déclarée pour usage externe.

```c
#include <stdio.h>

int global_var = 100;       // Durée persistante, linkage externe par défaut
static int static_var = 200; // Durée persistante, linkage interne

void demo() {
    auto int auto_var = 10;     // portée locale, durée automatique
    register int reg_var = 20;  // portée locale, durée automatique

    static int static_local = 0; // durée persistante, portée locale
    static_local++;
    printf("auto_var=%d, reg_var=%d, static_local=%d\n", auto_var, reg_var, static_local);
}

int main() {
    extern int global_var;
    printf("global_var=%d, static_var=%d\n", global_var, static_var);

    demo();
    demo();
    demo();

    return 0;
}
```

Sources utilisées

- Storage Classes in C - GeeksforGeeks
- Module 8 C - Storage Classes, UHCL
- Storage classes in C - Augustine Joseph - Medium
- Scope, Visibility and Lifetime of a Variable in C - Scaler Topics

Ce qu’il faut retenir

Ce cours synthétise les notions fondamentales de portée et durée de vie en C à travers les classes de stockage, afin de clarifier comment les variables sont gérées par le compilateur et exécutées en mémoire.

Qualificatifs `volatile` et `restrict`

- `volatile` et `restrict` : deux qualificatifs C spécifiques.
- `volatile` : Indique au compilateur de ne pas optimiser certaines variables, souvent liées au matériel ou à des contextes multithreading.
- `restrict` : Aide le compilateur à optimiser l'accès mémoire sous certaines conditions d'absence d'aliasing.
- Comprendre ces qualificatifs est essentiel pour écrire un code fiable et performant.

`volatile` : Interdire l'Optimisation du Compilateur

- **Définition :**
  - Prévient le compilateur que la valeur de la variable peut changer à tout moment **en dehors** du flux du programme (matériel, interruption, autre thread).
  - Le compilateur doit toujours relire la valeur depuis la mémoire et ne pas optimiser les accès.
- **Utilisations typiques :**
  - Accès aux registres matériels (ex: ports d'E/S).
  - Variables partagées entre threads (sans protection explicite).
  - Variables modifiées dans une interruption ou un signal.
- **Exemple Clé :**

```
volatile int flag = 0; // modifiée par une interruption
// ...
while(flag == 0) {
    // attente active, lecture forcée à chaque boucle
}
```

Sans `volatile`, le compilateur pourrait optimiser la boucle en supposant que `flag` ne change pas, menant à une boucle infinie.

`restrict` : Permettre l'Optimisation Mémoire

- **Définition (Standard C99) :**
  - Indique qu'un pointeur est le **seul moyen validé** d'accéder à la mémoire pointée durant sa portée.
  - Le compilateur peut alors optimiser en assumant qu'il n'y a pas d'aliasing (pas d'accès à la même mémoire via un autre pointeur).
- **Utilisations typiques :**
  - Fonctions manipulant des pointeurs où on peut garantir un accès non aliasé.
  - Optimisation des accès mémoire pour accélérer les boucles, calculs vectoriels, etc.
- **Exemple Clé :**

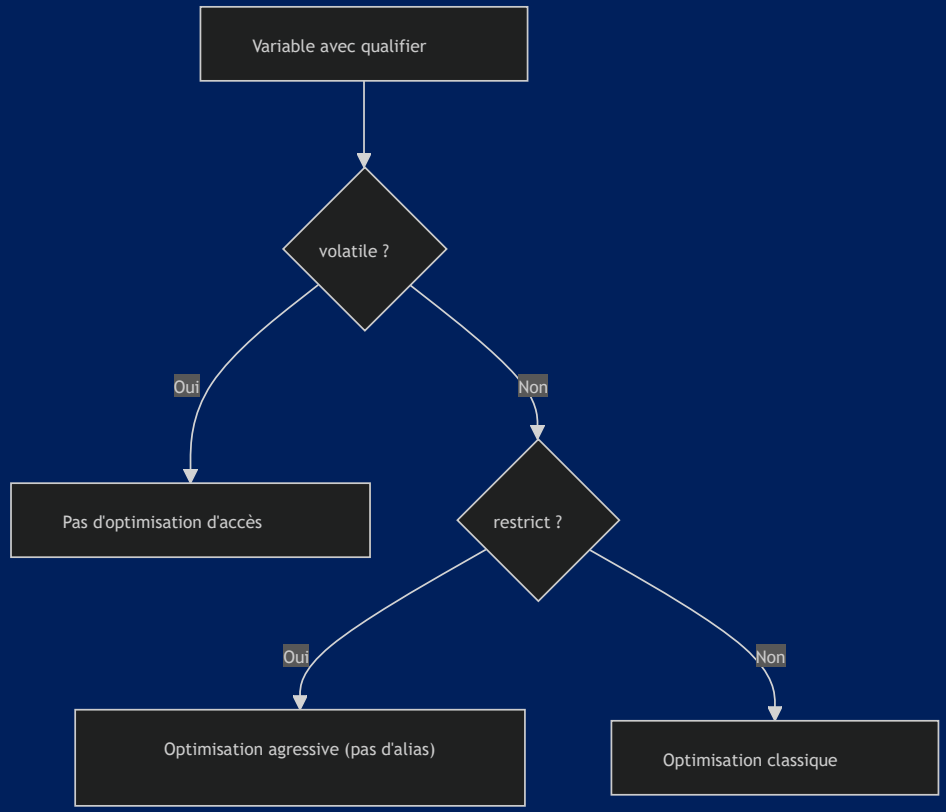
```c
void copy_arrays(int * restrict dest, const int * restrict src, size_t n) {
    for(size_t i = 0; i < n; i++) {
        dest[i] = src[i];
    }
}
```

`restrict` garantit ici que `dest` et `src` ne se chevauchent pas, permettant un code plus optimisé.

`volatile` vs `restrict` : Buts et Impacts

Qualificatif	But principal	Impact sur le compilateur
`volatile`	Interdire l’optimisation des accès mémoire car la variable peut changer de façon externe	Lecture/écriture forcée à chaque accès, minimise optimisation
`restrict`	Permettre au compilateur d’optimiser en précisant absence d’aliasing mémoire	Optimisations agressives possibles, moins de contraintes d’alias

Rôle des Qualificatifs & Notes Essentielles



- **Notes Complémentaires :**
  - `volatile` ne garantit pas la synchronisation entre threads : il faut des primitives spécifiques (mutex, atomic operations).
  - `restrict` doit être utilisé avec précaution : violer son hypothèse (aliasing réel) peut entraîner des comportements indéfinis.
  - Ces keywords sont orthogonaux et peuvent être combinés : `volatile int * restrict p;`

Ce qu’il faut retenir & Ressources

- **Points Clés :**
  - `volatile` et `restrict` sont des outils essentiels pour maîtriser la gestion mémoire.
  - Ils permettent d’optimiser le code et d’interagir avec le matériel ou des systèmes multi-threadés.
- **Sources Consultées :**
  - Qualifiers (const, volatile, restrict) en C - OpenClassrooms
  - Use of volatile and restrict Keywords - GeeksforGeeks
  - Understanding C’s volatile keyword - Embedded.com
  - ISO/IEC 9899:1999 - Programming languages — C

Rappels et Approfondissements : Les Standards C (C11, C17, C23)

Évolution du langage C : principales nouveautés (alignement, `_Generic`, threads C11)

Depuis la publication originelle du standard C (C89/90), le langage C a continué d'évoluer avec des mises à jour majeures : C11, C17, et C23. Ces standards ont introduit des fonctionnalités visant à améliorer la sécurité, la portabilité, la gestion de la concurrence et la flexibilité du langage, tout en conservant sa simplicité et sa performance.

Nous allons détailler ici quelques-unes des nouveautés clés introduites en C11, notamment :

- Les attributs d'alignement (`_Alignas` , `_Alignof`).
- Le mot-clé `_Generic` pour une forme de polymorphisme.
- La bibliothèque standard pour la gestion des threads.

Évolution du langage C : Standards récents

Standard	Année	Description courte
C11	2011	Ajout de la gestion des threads, `_Generic`, alignement mémoire avec `_Alignas` et `_Alignof`.
C17	2017	Correction de bugs, clarifications, pas de nouveautés majeures.
C23	à paraître / adoption progressive	Améliorations syntaxiques, fonctions supplémentaires, meilleur support Unicode, etc.

Alignement mémoire (`_Alignas` et `_Alignof`)

But de l'alignement L'alignement garantit qu'une variable est placée à une adresse mémoire multiple d'un certain nombre. Cela peut améliorer les performances d'accès et éviter des défauts matériels.

Nouveautés C11

- `_Alignas` : Permet de spécifier explicitement l'alignement d'une variable ou d'un type.
- `_Alignof` : Récupère l'alignement requis pour un type donné.

Exemple

```c
#include <stdio.h>
#include <stdalign.h> // depuis C11

struct S {
    char c;
    _Alignas(16) int x;
};

int main() {
    printf("Alignement de int : %zu\n", _Alignof(int));
    printf("Alignement de struct S : %zu\n", _Alignof(struct S));

    struct S s;
    printf("Adresse de s.x : %p\n", (void*)&s.x);
    return 0;
}
```

Le mot-clé `_Generic` : polymorphisme statique (C11)

Introduit en C11, `_Generic` permet d'écrire des macros qui choisissent un comportement ou une fonction selon le type de l'argument. Cela rapproche C d'un polymorphisme statique, similaire à ce qu'on trouve en C++.

Exemple d'utilisation

```c
#include <stdio.h>

#define print_type(x) _Generic((x), \
    int: printf("int: %d\n", x), \
    float: printf("float: %f\n", x), \
    double: printf("double: %f\n", x), \
    default: printf("Type inconnu\n"))

int main() {
    print_type(3);           // Affiche "int: 3"
    print_type(3.14f);       // Affiche "float: 3.140000"
    print_type(2.71828);     // Affiche "double: 2.718280"
    print_type("texte");     // Affiche "Type inconnu"
    return 0;
}
```

Ce code choisit la bonne fonction `printf` en fonction du type de la variable passée à la macro.

Threads en C11 : prise en charge standard

Bibliothèque `<threads.h>` C11 introduit une interface standardisée pour la création et la gestion de threads, de la synchronisation (mutex, variables condition) et d'autres primitives de concurrence.

Fonctions principales

- `thrd_create` : crée un nouveau thread.
- `thrd_join` : attend la fin d'un thread.
- `mtx_init`, `mtx_lock`, `mtx_unlock` : mutex.
- `cnd_wait`, `cnd_signal` : variables condition.

Exemple de création simple d’un thread

```c
#include <stdio.h>
#include <threads.h>

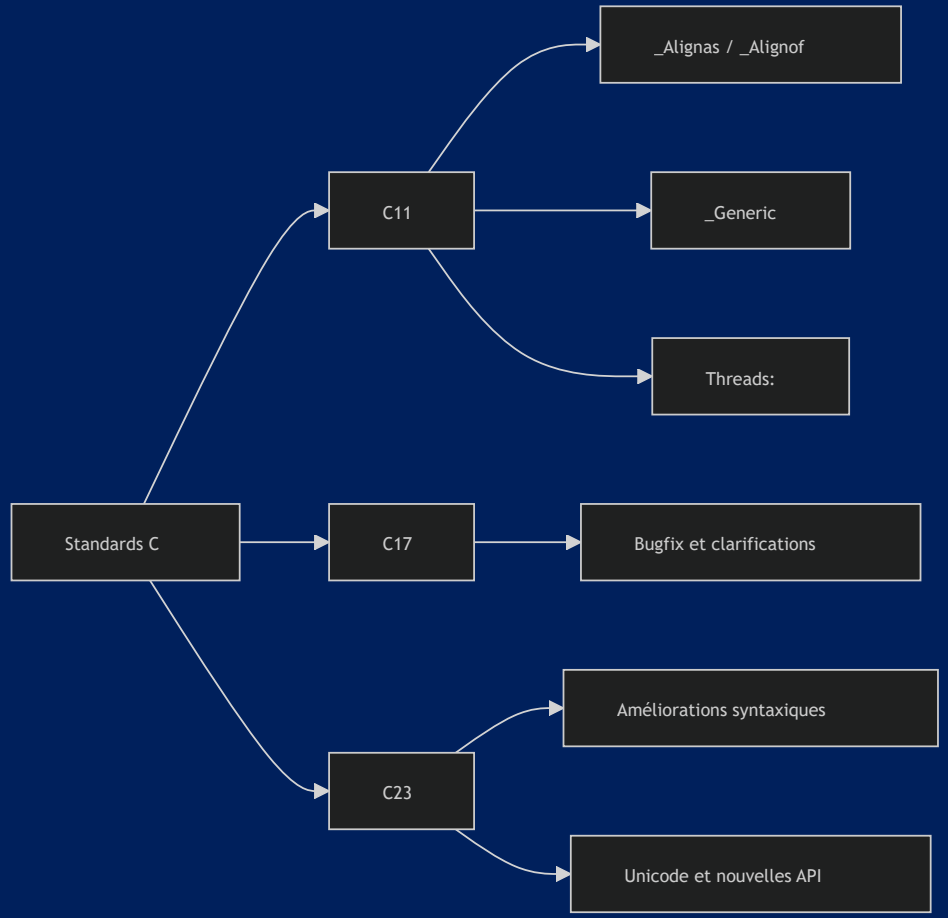
int thread_func(void *arg) {
    int *num = (int*)arg;
    printf("Hello from thread %d\n", *num);
    return 0;
}

int main() {
    thrd_t thread;
    int arg = 42;

    if (thrd_create(&thread, thread_func, &arg) ≠ thrd_success) {
        printf("Erreur création thread\n");
        return 1;
    }

    thrd_join(thread, NULL);
    return 0;
}
```

Points Clés et Ressources



Ce qu’il faut retenir Les évolutions modernes de C offrent aux développeurs des outils pour écrire du code plus robuste, performant et adapté aux environnements concurrents, tout en conservant l’esprit du langage.

Sources utilisées

- Documents officiels ISO C - Standard C11
- C11 Standard Library - cppreference
- Understanding C11 Threads - IBM Developer
- C11 _Generic explained - GeeksforGeeks
- C11 alignment - cppreference
- C23 draft and proposals - WG14

Rappels et Approfondissements : Les Standards C (C11, C17, C23)

Améliorations du mot-clé `const` en C23 et nouvelles fonctions de bibliothèque

Avec la standardisation récente du C23, le langage C a intégré plusieurs améliorations visant à renforcer la sécurité, la lisibilité du code et à enrichir la bibliothèque standard. Parmi celles-ci, des évolutions notables concernent l'utilisation du qualificatif `const` ainsi que l'ajout de nouvelles fonctions pratiques dans la bibliothèque.

`const` en C23 : Contexte et Améliorations

Problème antérieur

- Le mot-clé `const` indique qu'une variable ne doit pas être modifiée après son initialisation.
- Avant C23, son usage dans certains contextes (pointeurs composés, conversions) était limité ou source d'ambiguïtés.

Nouvelles règles de qualification `const`

- Le standard C23 introduit des règles plus souples et précises concernant la propagation de `const` et la conversion entre pointeurs qualifiés.
- **Bénéfices :**
  - Conversions plus sûres entre pointeurs `const` imbriqués (ex: `const int *` vers `const int * const *`).
  - Meilleure prise en charge des expressions constantes dans des contextes plus variés.

`const` en C23 : Exemple illustratif

```c
const int a = 10;
const int *p1 = &a;
const int * const *p2 = &p1;  // Conversion désormais mieux supportée en C23
```

Avant C23, certains compilateurs pouvaient rejeter cette conversion stricte à cause d'une qualification `const` "trop profonde". Les nouvelles règles en C23 apportent une flexibilité bienvenue et évitent des erreurs de compilation inutiles.

Nouvelles fonctions de la bibliothèque standard C23

Fonctions d'aide à la manipulation mémoire

- `memset_explicit` : Version de `memset` garantie non optimisable, utile pour effacer des zones sensibles (ex : clés cryptographiques) en évitant que le compilateur supprime ce nettoyage.

```
void *memset_explicit(void *ptr, int value, size_t num);
```

Fonctions temporelles

- Ajout de nouvelles fonctions pour une meilleure gestion du temps, incluant :
  - `timespec_getres()` : Récupérer la résolution d'un objet temporel.
  - Extensions pour faciliter le travail avec `struct timespec`.

Fonctions d'analyse de chaîne et conversion

- Quelques ajouts qui simplifient le traitement de texte et conversions dans `<string.h>` et `<stdlib.h>`.

`memset_explicit` : Un exemple concret

```c
#include <string.h>
#include <stdio.h>

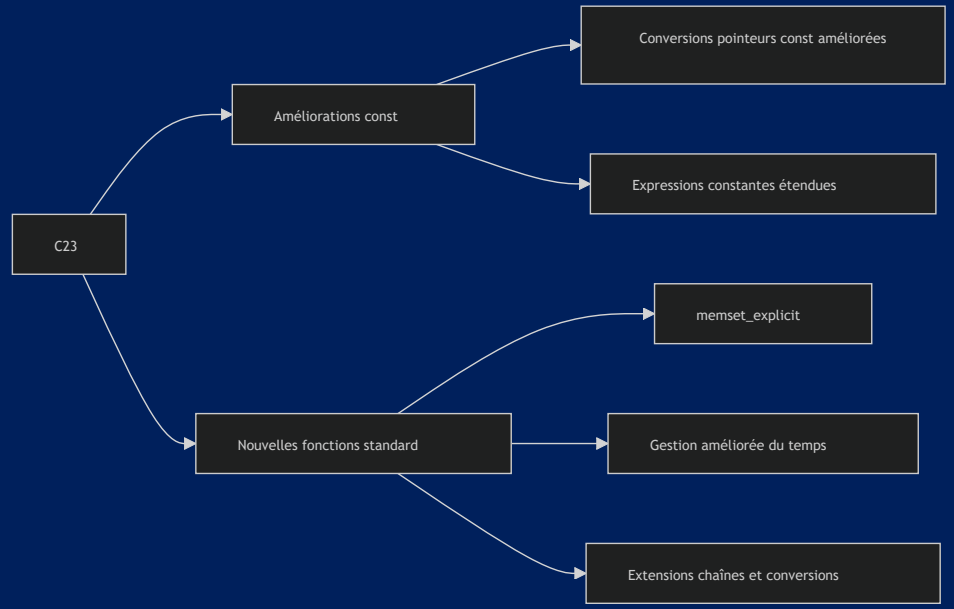
int main() {
    char password[16] = "secretPass123";
    printf("Mot de passe avant nettoyage : %s\n", password);

    memset_explicit(password, 0, sizeof(password));  // Nettoyage mémoire garanti

    printf("Mot de passe après nettoyage : %s\n", password);  // Affiche chaîne vide
    return 0;
}
```

Cette fonction s'assure que le compilateur ne supprimera pas l'appel à `memset`, contrairement à `memset` classique dont l'appel pourrait être optimisé ou supprimé si la variable n'est plus utilisée ensuite, un aspect crucial pour la sécurité des données sensibles.

Synthèse et Ressources



Cette présentation met en lumière les avancées majeures introduites dans le standard C23 pour la gestion améliorée de `const` — offrant plus de flexibilité et de sécurité dans le typage — ainsi que les extensions fonctionnelles en bibliothèque qui facilitent le développement moderne.

Sources utilisées :

- ISO/IEC 9899:2023 Draft
- C23 new features - GitHub: gcc wiki
- memset_explicit introduction - Microsoft Documentation
- Exploring the new features of C23 - Embedded Artistry
- Stack Overflow - Discussion sur const et C23

Gestion Avancée de la Mémoire : Allocation Dynamique

Rappels et Erreurs Courantes (fuites mémoire, double free)

Introduction

La gestion dynamique de la mémoire en C permet d'allouer et libérer la mémoire au moment de l'exécution (runtime), par opposition à la mémoire statique ou automatique. Elle utilise les fonctions standards `malloc`, `calloc`, `realloc` et `free`.

Une mauvaise utilisation peut entraîner des problèmes classiques :

- **Fuites mémoire** : mémoire non libérée.
- **Double free** : libération multiple d'un même bloc.

Ce cours synthétise les principes essentiels pour éviter ces erreurs.

Principales fonctions d’allocation dynamique

Fonction	Description	Particularités
`malloc`	Alloue un bloc mémoire brut de taille indiquée (non initialisée).	Retourne un pointeur `void*` sur la mémoire allouée, ou `NULL` si échec.
`calloc`	Alloue un bloc pour `n` éléments de taille `size` et initialise à 0.	Plus sûr si besoin d’un buffer initialisé à zéro.
`realloc`	Redimensionne un bloc mémoire précédemment alloué.	Peut déplacer le bloc, retourne un nouveau pointeur, `NULL` si échec.
`free`	Libère un bloc mémoire précédemment alloué.	Ne supprime pas la variable, juste la mémoire.

Utilisation et Exemples : `malloc` et `free`

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *tab = (int *)malloc(5 * sizeof(int));
    if (tab == NULL) {
        perror("malloc échoué");
        return 1;
    }
    for (int i = 0; i < 5; i++) {
        tab[i] = i * i;
        printf("%d ", tab[i]);
    }
    printf("\n");

    free(tab); // Libération de la mémoire
    return 0;
}
```

Utilisation et Exemples : `calloc` et `realloc`

`calloc`

```c
int *tab_zero = (int *)calloc(5, sizeof(int));
if (tab_zero == NULL) {
    perror("calloc échoué");
    return 1;
}
// tab_zero est initialisé à zéro
free(tab_zero);
```

`realloc`

```c
int *tab_resize = (int *)malloc(3 * sizeof(int));
// ... initialisation ...
tab_resize = (int *)realloc(tab_resize, 6 * sizeof(int));
if (tab_resize == NULL) {
    perror("realloc échoué");
    // Attention: éviter fuite mémoire si realloc échoue
}
// ... initialiser nouvelle zone ...
free(tab_resize);
```

Erreurs Courantes et Cycle de Vie

1. Fuites mémoire

Oubli de `free` dans un programme de longue durée = consommation excessive mémoire.

```
int *p = malloc(10 * sizeof(int));
// utilisation mais pas de free → FUITE MÉMOIRE
```

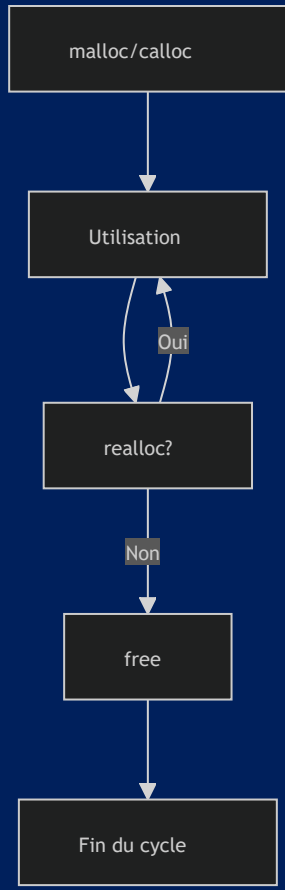
Remarque : un programme court qui termine libère généralement la mémoire, mais ce n'est pas une bonne pratique.

2. Double free

Libération d'un même bloc plusieurs fois = comportement indéfini et souvent crash.

```
int *p = malloc(10 * sizeof(int));
free(p);
// free(p); // ERREUR : double free
```

Diagramme : Cycle de vie d’un bloc mémoire



Bonnes Pratiques & Conclusion

Bonnes pratiques

- Toujours vérifier le résultat de `malloc/calloc/realloc` (!= NULL).
- Initialiser la mémoire après allocation si nécessaire (sauf avec `calloc`).
- Après `free(ptr)`, il est recommandé de faire `ptr = NULL` pour éviter un double free accidentel.
- Éviter de réassigner directement le résultat de `realloc` :

```
int *tmp = realloc(ptr, new_size);
if (tmp != NULL) { ptr = tmp; }
else { /* gestion erreur, ptr reste valide */ }
```

- Utiliser des outils de détection des fuites mémoire (Valgrind, AddressSanitizer).

Sources utilisées

- malloc, calloc, realloc, free - cppreference
- Gestion mémoire en C - OpenClassrooms
- Common Memory Management Errors in C - GeeksforGeeks
- Valgrind Documentation

Ce cours rassemble les mécanismes fondamentaux de la gestion dynamique mémoire en C, ainsi que les erreurs typiques à éviter pour garantir la stabilité et l'efficacité des programmes.

Gestion Avancée de la Mémoire : Allocation Dynamique

Sécuriser vos allocations C avec `malloc`, `calloc`, `realloc`, `free`

L’allocation dynamique de mémoire est essentielle en C pour gérer des données de taille variable. Une gestion efficace repose sur l’utilisation correcte des fonctions d’allocation et l’adoption de stratégies robustes.

Il est crucial de sécuriser le programme par la vérification systématique des retours d’allocation et des techniques adaptées pour limiter la fragmentation et optimiser la mémoire.

Stratégies d’Allocation Mémoire

Lorsqu’une zone mémoire doit évoluer, il est courant d’allouer une capacité initiale, puis de l’agrandir via `realloc` selon les besoins.

Stratégie classique : Doublement progressif On multiplie généralement la capacité par 2 à chaque saturation. Cela limite les appels coûteux à `realloc` (copies de données).

```c
size_t capacity = 4;
size_t size = 0; // éléments utilisés
int *array = malloc(capacity * sizeof(int));
if (array == NULL) { /* gestion d'erreur */ }

for (int i = 0; i < 10; i++) {
    if (size == capacity) {
        size_t new_capacity = capacity * 2;
        int *tmp = realloc(array, new_capacity * sizeof(int));
        if (tmp == NULL) {
            // Erreur : libérer array, exit
            free(array);
            exit(EXIT_FAILURE);
        }
        array = tmp;
        capacity = new_capacity;
    }
    array[size++] = i;
}
```

Allocation par Blocs ou Pools

Pour des allocations répétées d’objets de taille fixe, préallouer une zone (pool) réduit la fragmentation et augmente la rapidité, souvent dans les systèmes embarqués.

Vérification Obligatoire des Retours d'Allocation

Pourquoi vérifier ?

Un échec d'allocation (`malloc` , `calloc` , `realloc`) se traduit par un retour de `NULL` . Utiliser ce pointeur `NULL` sans contrôle entraîne un comportement indéfini, potentiellement un plantage.

Comment vérifier ?

- **Toujours tester** que le pointeur retourné n'est pas `NULL` avant d'utiliser la mémoire.
- En cas d'erreur, **libérer les ressources** et sortir proprement ou prévoir un plan de secours.

Exemple simple

```
int *ptr = malloc(100 * sizeof(int));
if (ptr == NULL) {
    fprintf(stderr, "Erreur : allocation mémoire échouée\n");
    exit(EXIT_FAILURE); // Quitter le programme proprement
}
// Utilisation de ptr
free(ptr); // Libérer la mémoire après usage
```

Vérification Spécifique pour `realloc`

`realloc` peut retourner `NULL` tout en **conservant le pointeur original valide**.

Erreur à éviter :

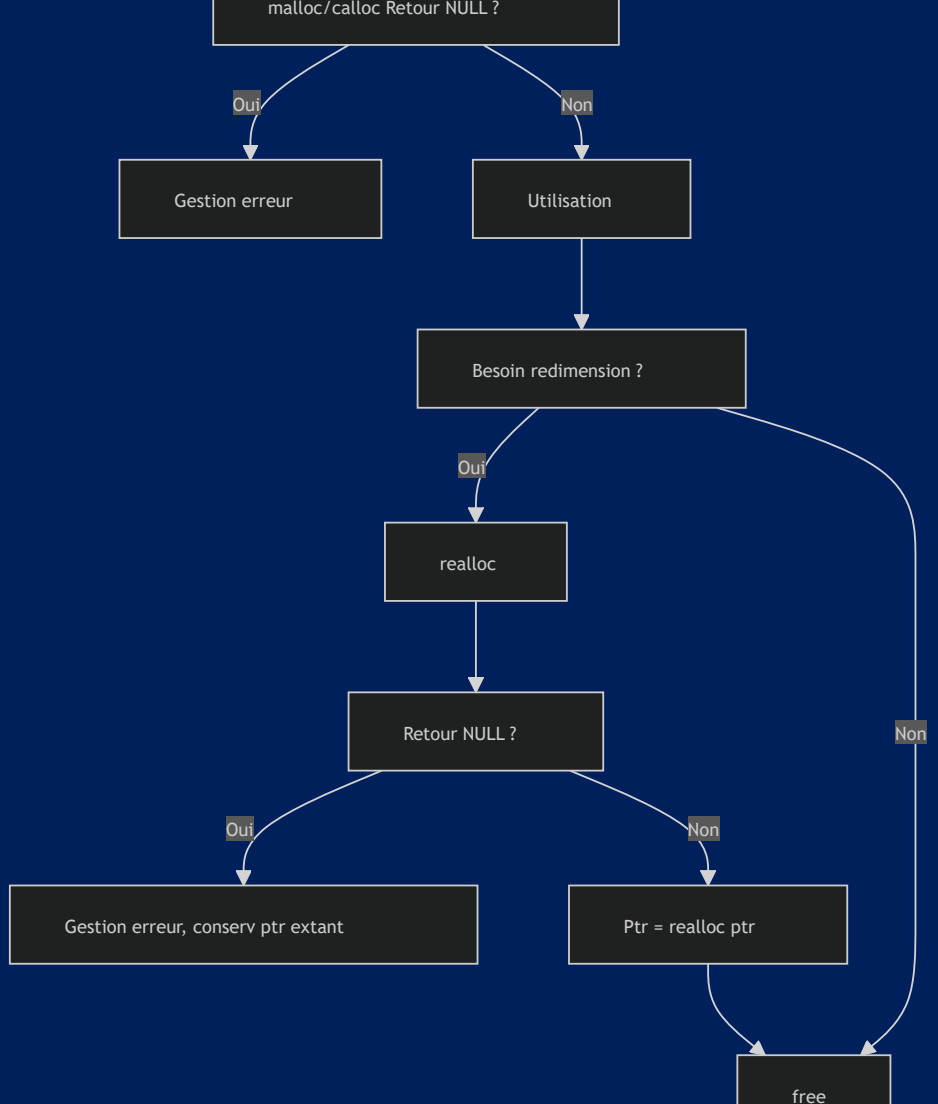
```
ptr = realloc(ptr, new_size); // RISQUE de FUITE si realloc retourne NULL
```

Si `realloc` échoue et retourne `NULL`, la référence à l'ancienne zone mémoire (initialement pointée par `ptr`) est perdue, créant une fuite mémoire car `ptr` est écrasé.

Méthode correcte et sécurisée :

```
int *tmp = realloc(ptr, new_size);
if (tmp == NULL) {
    // Gestion d'erreur
    // ATTENTION : ptr est toujours valide et pointe vers l'ancienne zone !
    // Vous pouvez continuer à utiliser ptr ou le libérer si l'échec est critique.
} else {
    ptr = tmp; // Seulement si realloc a réussi
}
```

Cette approche garantit que la mémoire originale est toujours accessible même si le redimensionnement échoue.


malloc/calloc Retour NULL ?
Oui
Non
Gestion erreur
Utilisation
Besoin redimension ?
Oui
Non
realloc
Retour NULL ?
Oui
Non
Gestion erreur, conserv ptr extant
Ptr = realloc ptr
free

Bonnes pratiques complémentaires

- **Initialiser** les pointeurs alloués (avec `calloc` ou manuellement).
- **Libérer toujours** la mémoire allouée (`free`) pour éviter les fuites.
- Documenter et **centraliser** la gestion mémoire si possible.
- Utiliser des **outils d'analyse dynamique** (Valgrind, AddressSanitizer) pour détecter erreurs et fuites.

Points Clés & Ressources

Ce qu’il faut retenir

Ce cours a exposé des stratégies d’allocation efficaces et sécurisées, tout en insistant sur la nécessité de vérifier systématiquement les retours d’appel d’allocation mémoire pour garantir la robustesse des programmes en C.

Sources

- Dynamic memory allocation in C - GeeksforGeeks
- C malloc and realloc best practices - Stack Overflow
- Memory Management in C - OpenClassrooms
- Valgrind documentation - Memcheck

Gestion Avancée de la Mémoire :
Pointeurs de Fonctions et `void*`

Introduction

Les pointeurs en C ne servent pas qu'à référencer des données. Ils permettent aussi de :

- Manipuler des fonctions via des **pointeurs de fonctions**.
- Gérer des adresses de mémoire de type inconnu avec des **pointeurs génériques (`void*`)**.

Ces concepts sont essentiels pour la flexibilité et la puissance du langage C. Ce cours vise à clarifier leur syntaxe, leur usage pratique et les précautions nécessaires.

Pointeurs de Fonctions : Définition et Syntaxe

Définition

- Variable qui stocke l'adresse d'une fonction.
- Permet d'appeler la fonction via ce pointeur.
- Mécanisme puissant pour :
  - Callbacks
  - Tables de dispatch dynamiques
  - Polymorphisme limité

Syntaxe de Déclaration

- Pointeur vers une fonction qui prend un `int` et retourne un `int` :

```
int (*ptr_func)(int);
```

  - Les parenthèses autour de `*ptr_func` sont nécéssaires pour indiquer que `ptr_func` est un pointeur.

Pointeurs de Fonctions : Exemple et Usages Typiques

Exemple Simple

```c
#include <stdio.h>

int carre(int x) {
    return x * x;
}

int main() {
    int (*fptr)(int) = carre;  // Affectation de l'adresse
    int val = 5;
    printf("Carré de %d = %d\n", val, fptr(val)); // Appel via pointeur

    return 0;
}
```

Usages Typiques

- **Callbacks** dans des API (ex : fonctions de comparaison pour `qsort`).
- **Tables de fonctions dynamiques** (ex : menu interactif).
- **Implémentations de machines à états**.

Pointeurs Génériques (`void*`) : Définition et Exemple

Définition

- `void*` est un pointeur **sans type**.
- Peut pointer vers n'importe quel type de donnée.
- Utilisé pour écrire des fonctions et structures génériques en C.

Conversion Implicite et Explicite

- Un pointeur typé peut être converti **implicitement** en `void*` .
- Un `void*` doit être **casté explicitement** en pointeur du type attendu avant utilisation.

Exemple

```c
void affiche_int(void *ptr) {
    int *iptr = (int *)ptr; // Cast explicite nécessaire
    printf("%d\n", *iptr);
}

int main() {
    int a = 42;
    affiche_int(&a); // Conversion implicite de &a (int*) en void*
    return 0;
}
```

Usages Avancés et Précautions

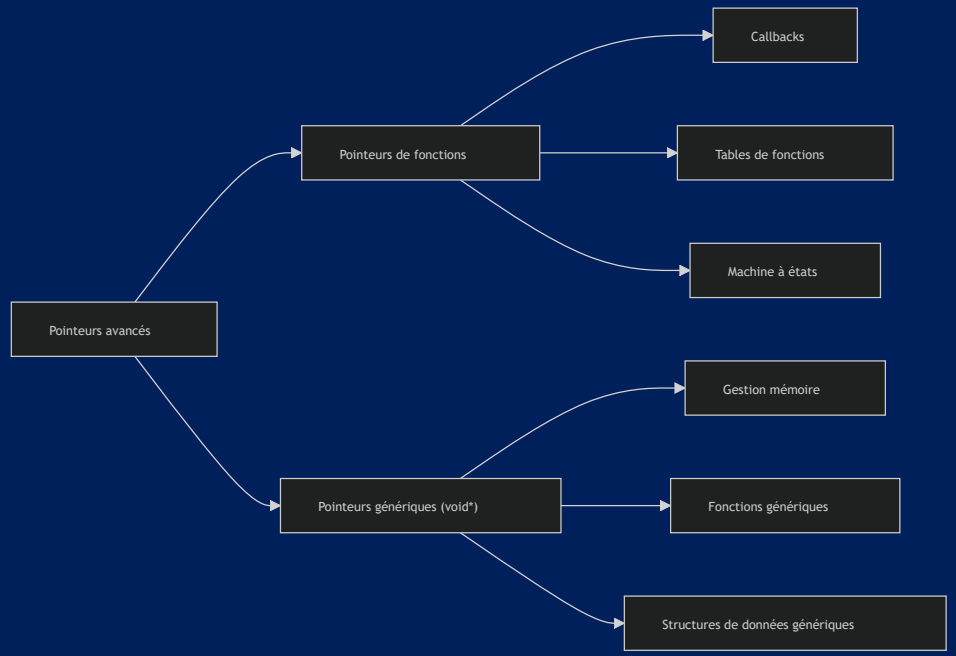
Utilisations Courantes de `void*`

- Fonctions génériques de gestion mémoire (`malloc` retourne `void*`).
- APIs génériques (`qsort` , `bsearch`).
- Stockage générique dans des structures de données (listes chaînées).

Relation entre les deux types de pointeurs

- Il n'est **pas possible** de convertir directement un pointeur de fonction en `void*` (ou inversement) sans provoquer un comportement indéfini. Ces pointeurs sont distincts.

Diagramme : Usages des pointeurs avancés



Précautions et Bonnes Pratiques

- Assurez-vous que le pointeur de fonction pointe vers une fonction compatible avec la signature attendue.
- Ne pas faire de conversions entre pointeurs de fonction et `void*`.
- Avec `void*`, le cast explicite est obligatoire avant l'accès.
- En multithread, garantissez la synchronisation si les pointeurs de fonctions manipulent des données globales modifiables.

Ce qu’il faut retenir et Ressources

En bref : Les pointeurs de fonctions et les pointeurs génériques (`void*`) sont des composants indispensables en C pour une programmation flexible, modulaire et générique. Leur maîtrise permet d’écrire des codes plus adaptables et puissants.

Sources utilisées :

- Pointer to function in C - GeeksforGeeks
- Void pointer in C - Tutorialspoint
- ISO/IEC 9899:2018 - Standard C
- The GNU C Library - Function pointers
- Wikipedia - Void pointer

Gestion Avancée de la Mémoire

Pointeurs Avancés : Tableaux de Pointeurs et Structures

La manipulation de tableaux de pointeurs et de pointeurs pointant sur des structures en C permet une gestion flexible et performante des données complexes. C’est essentiel pour créer des tableaux dynamiques d’objets, gérer des collections hétérogènes, ou construire des structures de données efficaces.

Ce cours explore les concepts, la syntaxe et les bonnes pratiques liées à ces usages majeurs.

1. Les Tableaux de Pointeurs

Concept & Syntaxe Un tableau de pointeurs est un tableau dont les éléments sont des pointeurs.

- **Exemple simple (statique) :**

```
char *jours[] = {"Lundi", "Mardi", "Mercredi", "Jeudi", "Vendredi"};
// jours est un tableau de 5 pointeurs vers des chaînes constantes.
```

Utilisation Dynamique Créer un tableau de pointeurs dont les éléments (les chaînes ici) sont alloués dynamiquement.

- **Méthode :** `malloc` pour le tableau de pointeurs, puis `strdup` pour chaque chaîne.
- **Exemple :**

```
char **noms = malloc(n * sizeof(char*)); // Tableau de pointeurs
noms[0] = strdup("Alice"); // Allocation pour chaque chaîne
//  ...
// N'oubliez pas de libérer chaque élément (noms[i]) avant de libérer le tableau (noms).
```

2. Les Pointeurs vers Structures

Déclaration et Accès Permet de manipuler des instances de structures de manière indirecte.

- **Exemple :**

```c
struct Point { int x, y; };
struct Point p = {10, 20};
struct Point *ptr = &p; // ptr pointe sur p

// Accès aux membres via le pointeur
printf("Coordonnées : (%d, %d)\n", ptr->x, ptr->y);
// On utilise l'opérateur "->" pour accéder aux membres via un pointeur.
```

Allocation Dynamique de Structures Créer une structure sur le tas, utile pour des objets de durée de vie variable.

- **Fonction type :**

```c
struct Point *creer_point(int x, int y) {
    struct Point *p = malloc(sizeof(struct Point)); // Allocation
    if (p != NULL) {
        p->x = x; p->y = y;
    }
    return p;
}
// N'oubliez pas de free(p) après utilisation.
```

3. Tableaux de Pointeurs vers Structures

Collections Dynamiques d'Objets Combine les deux concepts précédents pour gérer des collections de structures allouées dynamiquement.

- **Exemple :** Un tableau de pointeurs `struct Point *`

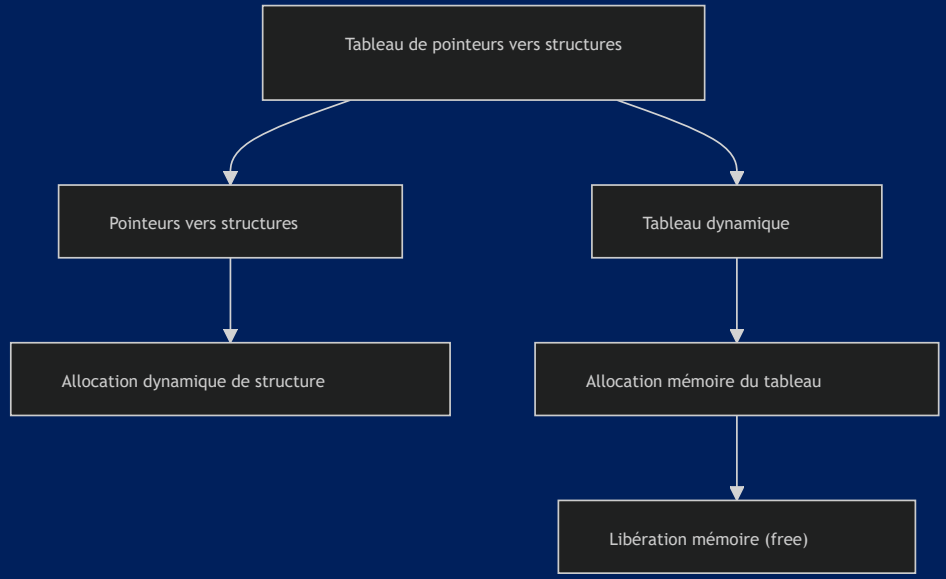
```c
int n = 2;
struct Point **tab_points = malloc(n * sizeof(struct Point*));
// tab_points est un tableau de pointeurs vers des structures Point

tab_points[0] = creer_point(1, 2); // Chaque élément pointe vers une structure allouée
tab_points[1] = creer_point(3, 4);

// Accès : tab_points[i]→x

// Libération :
for (int i = 0; i < n; i++) {
    free(tab_points[i]); // Libérer chaque structure
}
free(tab_points); // Libérer le tableau de pointeurs
```

4. Interactions Pointeurs et Structures



Ce diagramme illustre la relation hiérarchique et les dépendances des concepts abordés pour la gestion de structures dynamiques via des pointeurs.

5. Bonnes Pratiques & Pour Aller Plus Loin

Bonnes Pratiques Essentielles

- **Vérifiez toujours :** Le résultat des `malloc` et `strdup` doit être contrôlé (`≠ NULL`).
- **Libération mémoire :** Assurez-vous de libérer *toutes* les zones mémoire allouées (structures, chaînes, tableaux).
- **Opérateurs :** Utilisez `→` pour les membres via un pointeur et `.` pour une structure directe.
- **Modularité :** Pour des tableaux dynamiques complexes, créez des fonctions dédiées à leur création et destruction pour garantir la cohérence mémoire.

Ce qu'il faut retenir Ce cours clarifie l'usage des tableaux de pointeurs et des pointeurs sur structures afin de manipuler efficacement des données complexes, dynamiques et extensibles en C.

Sources Utilisées

- Using pointers to structs - GeeksforGeeks
- Arrays of pointers - C Programming
- Dynamic array of pointers - Stack Overflow
- C Structures and Pointers - Tutorialspoint
- ISO/IEC 9899:2018 Standard C

Listes Chaînées, Piles et Files

Introduction aux Structures Dynamiques

Les listes chaînées, piles et files sont des structures de données fondamentales pour la gestion dynamique des collections d’éléments en mémoire. Elles permettent une flexibilité et une efficacité accrues par rapport aux tableaux statiques.

Cette présentation synthétique explore leurs caractéristiques, fonctionnements et offre des exemples d’implémentation simple en C.

Listes Chaînées : La Base Dynamique

Définition

Une **liste chaînée** est une structure de données composée d'éléments (nœuds). Chaque nœud contient une donnée et un pointeur vers le nœud suivant. Elles permettent des insertions/suppressions efficaces n'importe où dans la liste, contrairement aux tableaux.

Structure Type en C

```c
typedef struct Node {
    int value;
    struct Node *next;
} Node;
```

Clé : Chaque maillon ("Node") se lie au suivant.

Listes Chaînées : Opérations Essentielles

Insertion en Tête

L’insertion d’un nouvel élément en début de liste est une opération rapide.

```
Node *inserer_en_tete(Node *head, int val) {
    Node *new_node = malloc(sizeof(Node));
    if (new_node == NULL) return head; // Gestion erreur
    new_node->value = val;
    new_node->next = head; // Le nouveau nœud pointe vers l'ancienne tête
    return new_node;       // Le nouveau nœud devient la tête
}
```

Parcours de la Liste

Pour afficher ou traiter chaque élément.

```
void afficher_liste(Node *head) {
    Node *current = head;
    while (current ≠ NULL) {
        printf("%d → ", current→value);
        current = current→next;
    }
    printf("NULL\n");
}
```

Piles (Stack) : La Logique LIFO

Définition

Une **pile** suit une logique **Last In, First Out (LIFO)** : le dernier élément inséré est le premier à être extrait. Imaginez une pile d'assiettes.

Implémentation Simplifiée (avec liste chaînée)

Les piles sont souvent implémentées en utilisant des listes chaînées, l'insertion et la suppression se faisant à la même extrémité (la "tête" de la liste).

```
typedef Node Stack; // Une pile est simplement un pointeur vers un nœud

Stack *push(Stack *s, int val) { // Empiler
    return inserer_en_tete(s, val);
}

Stack *pop(Stack *s, int *val) { // Dépiler
    if (s == NULL) return NULL; // Pile vide
    *val = s->value;
    Node *temp = s->next;
    free(s); // Libère la mémoire de l'élément dépilé
    return temp;
}
```

Files (Queue) : La Logique FIFO

Définition

Une **file** respecte la logique **First In, First Out (FIFO)** : le premier élément inséré est le premier à être extrait. Comme une file d’attente réelle.

Implémentation Simplifiée (avec liste chaînée)

Pour une file, les insertions se font à la fin et les extractions au début. Cela nécessite de gérer à la fois la "tête" (`front`) et la "queue" (`rear`).

```c
typedef struct Queue {
    Node *front; // Tête de la file
    Node *rear;  // Fin de la file
} Queue;

void enqueue(Queue *q, int val) { // Ajouter à la fin
    Node *new_node = malloc(sizeof(Node)); //  ... (code d'insertion)
    //  ... Le nouveau nœud est ajouté après 'q->rear'
    //  ... et 'q->rear' est mis à jour.
}

int dequeue(Queue *q, int *val) { // Retirer du début
    if (q->front == NULL) return 0; // File vide
    Node *temp = q->front;
    *val = temp->value;
    q->front = q->front->next; // La nouvelle tête devient l'élément suivant
    if (q->front == NULL) q->rear = NULL; // Si la file est vide après, maj 'rear'
    free(temp); // Libère la mémoire
    return 1;
}
```

Synthèse & Ressources Utiles

Ce qu’il faut retenir

- Les listes chaînées facilitent les insertions/suppressions en temps constant (O(1)) en tête.
- Piles et files sont des abstractions mais souvent implémentées via listes chaînées ou tableaux.
- Gérer correctement la libération de mémoire est crucial dans les structures dynamiques.
- Choisir la structure selon le besoin spécifique d’ordre d’accès (LIFO vs FIFO).

Sources utilisées

- Linked List in C - GeeksforGeeks
- Stack Data Structure in C - Programiz
- Queue Data Structure in C - GeeksforGeeks
- CS50 Lecture - Data Structures
- Tutorialspoint - Linked List

Structures de Données Avancées

Listes, Piles, Files : Implémentations efficaces

Introduction

Les listes doublement chaînées et les buffers circulaires sont des structures de données optimisées pour des opérations fréquentes d’insertion, suppression et gestion mémoire circulaire.

Elles améliorent la flexibilité par rapport aux listes simplement chaînées et aux files classiques en offrant une meilleure gestion des accès et des performances.

1. Listes doublement chaînées (Doubly Linked Lists)

Concept : Séquence de nœuds où chaque nœud possède deux pointeurs : un vers l'élément suivant (`next`) et un vers l'élément précédent (`prev`).

Cela permet un parcours bidirectionnel et simplifie la suppression.

Structure de nœud :

```
typedef struct Node {
    int value;
    struct Node *prev;
    struct Node *next;
} Node;
```

Avantages :

- Parcours en avant et en arrière possible.
- Suppression et insertion en O(1) sans nécessiter de parcourir la liste pour trouver le prédécesseur.
- Utile pour les piles, files, et autres structures complexes (ex: deque).

Listes doublement chaînées : Opérations clés

Exemple d'insertion en début :

```c
Node* insert_head(Node *head, int val) {
    Node *new_node = malloc(sizeof(Node));
    if (!new_node) return head;
    new_node->value = val;
    new_node->prev = NULL;
    new_node->next = head;
    if (head) head->prev = new_node;
    return new_node;
}
```

Exemple de suppression d'un nœud :

```c
Node* delete_node(Node *head, Node *node) {
    if (!head || !node) return head;
    if (node->prev) node->prev->next = node->next;
    else head = node->next;
    if (node->next) node->next->prev = node->prev;
    free(node);
    return head;
}
```

2. Buffers circulaires (Circular Buffers)

Concept : Structure de données en mémoire utilisée comme une file circulaire. Une fois la fin du buffer atteinte, l'écriture ou la lecture reprend au début, évitant un coûteux décalage des valeurs.

Caractéristiques :

- Taille fixe (statique ou dynamique).
- Deux indices : `head` (point d'écriture), `tail` (point de lecture).
- Gestion des cas de buffer plein ou vide par des contrôles spécifiques.

Structure typique :

```c
typedef struct {
    int *buffer;
    size_t head;
    size_t tail;
    size_t max;    // capacité du buffer
    int full;      // indicateur buffer plein
} CircularBuffer;
```

Buffers circulaires : Opérations clés

Exemple d'insertion (enqueue) :

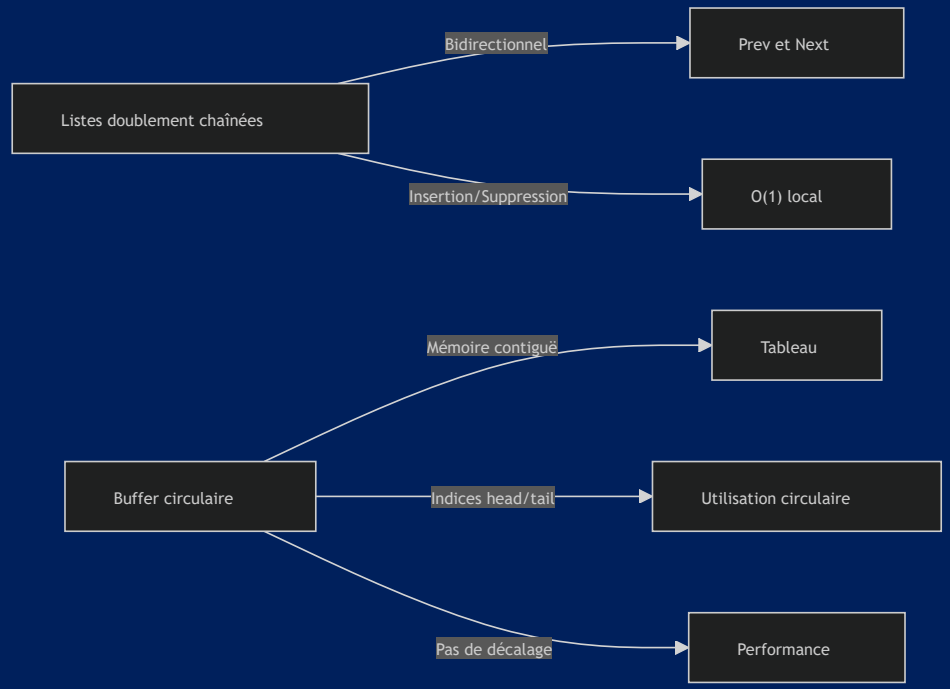
```c
void cb_enqueue(CircularBuffer *cb, int val) {
    if (cb->full) {
        // Optionnel : écraser le plus ancien ou gestion d'erreur
        cb->tail = (cb->tail + 1) % cb->max;
    }
    cb->buffer[cb->head] = val;
    cb->head = (cb->head + 1) % cb->max;
    cb->full = (cb->head == cb->tail);
}
```

Exemple de lecture (dequeue) :

```c
int cb_dequeue(CircularBuffer *cb, int *val) {
    if (cb->head == cb->tail && !cb->full) return 0; // buffer vide
    *val = cb->buffer[cb->tail];
    cb->tail = (cb->tail + 1) % cb->max;
    cb->full = 0;
    return 1;
}
```

Comparaison, usages et ce qu’il faut retenir

Comparaison des structures :



Résumé des usages :

Structure	Avantages	Usages typiques
Listes doublement chaînées	Parcours bidirectionnel, suppression rapide	Implémentation de déques, gestion d'historique
Buffers circulaires	Utilisation fixe de mémoire, performances optimales	Communication en temps réel, buffers audio/vidéo

Ce qu'il faut retenir : Ce cours présente deux structures performantes permettant un contrôle efficace des collections grâce à une gestion mémoire optimisée et adaptée aux besoins d'accès rapide et modifiable.

Sources utilisées :

- Doubly Linked List - GeeksforGeeks
- Circular Buffer - Embedded Artistry
- Circular Queue Implementation - GeeksforGeeks
- [Data Structures and Algorithms in C, Adam Drozdek]
- Coding Labs - Circular Buffer in C

Structures de Données Avancées

Arbres Binaires de Recherche et Arbres Équilibrés

- **Introduction aux ABR**
  - Permettent de stocker, rechercher, insérer et supprimer efficacement des données ordonnées.
  - Visent des performances optimales, idéalement en temps logarithmique (O(log n)).
  - Nécessitent des **arbres équilibrés** pour éviter la dégradation en liste chaînée (O(n)).
- **Définition d’un ABR**
  - Chaque nœud respecte la propriété d’ordre :
    - Éléments du sous-arbre gauche < Valeur du nœud.
    - Éléments du sous-arbre droit > Valeur du nœud.
  - Facilite une recherche rapide par parcours directionnel.

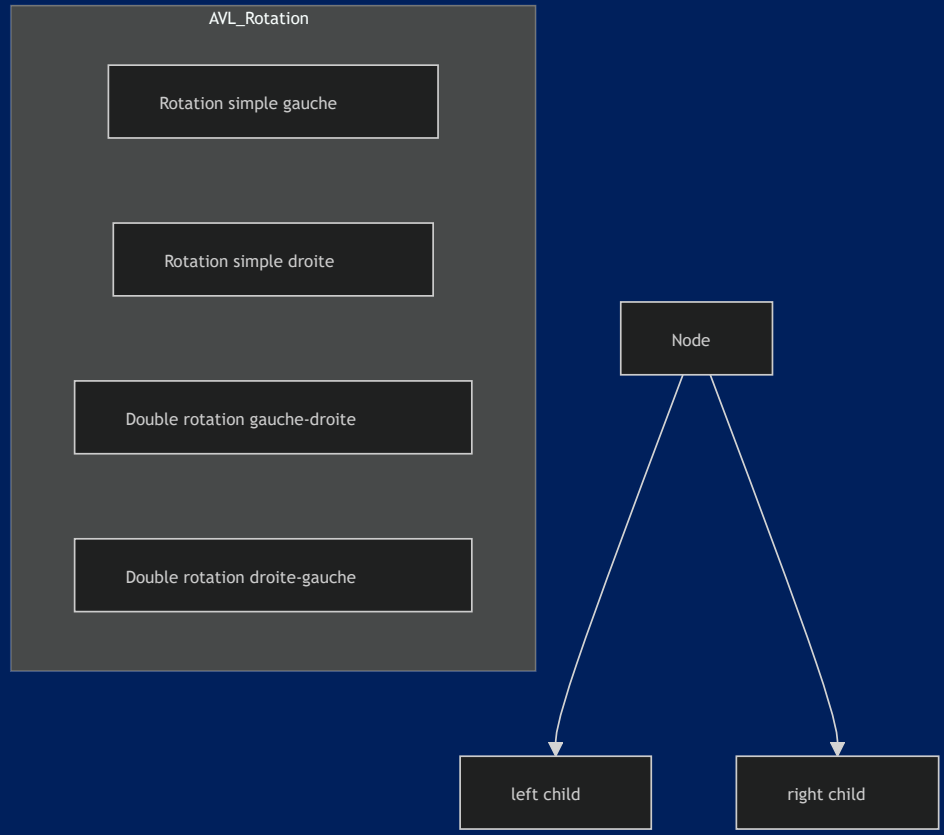
- **Structure d’un nœud (exemple en C)**

```c
typedef struct Node {
    int key;
    struct Node *left;
    struct Node *right;
} Node;
```

- **Opérations clés : Recherche et Insertion**
  - La recherche suit une logique récursive, se dirigeant vers le sous-arbre gauche ou droit selon la valeur recherchée.
  - L’insertion trouve la position appropriée en respectant la propriété d’ordre, puis crée un nouveau nœud.

L’impératif de l’Équilibrage : Maintenir la performance

- **Pourquoi équilibrer ?**
  - Un ABR classique peut devenir déséquilibré, ressemblant alors à une liste chaînée.
  - Dans ce cas, les opérations passent de O(log n) à O(n), perdant leur efficacité.
  - Un arbre équilibré maintient une hauteur minimale pour garantir des performances O(log n).
- **Principaux types d’arbres équilibrés**
  - **Arbre AVL**
    - La différence de hauteur entre le sous-arbre gauche et droit de chaque nœud ne dépasse jamais 1 (facteur d’équilibre ∈ {-1, 0, 1}).
    - Des **rotations** (simples ou doubles) sont effectuées automatiquement pour corriger les déséquilibres lors des insertions/suppressions.
  - **Arbre Rouge-Noir**
    - Moins strict que l’AVL, plus couramment utilisé.
    - Les nœuds sont colorés en rouge ou noir selon des règles spécifiques.
    - Ces règles garantissent un équilibre suffisant sans être aussi rigoureuses que l’AVL.



- **Structure d’un ABR :** Chaque nœud pointe vers ses enfants gauche et droit, formant une hiérarchie ordonnée.
- **Rotations AVL :** Ces opérations restructurent l’arbre localement pour rétablir son équilibre après une modification, assurant ainsi la propriété AVL.

Complexité des Opérations : Impact de l’équilibrage

L’équilibrage est crucial pour la performance des structures de données.

Opération	ABR non équilibré	Arbres équilibrés (AVL, Rouge-Noir)
Recherche	O(n)	O(log n)
Insertion	O(n)	O(log n)
Suppression	O(n)	O(log n)

- **ABR non équilibré :** Dans le pire des cas (données déjà triées), l’arbre dégénère en liste chaînée, et les opérations deviennent linéaires.
- **Arbres équilibrés :** Grâce à leurs mécanismes d’auto-équilibrage, ils garantissent des performances logarithmiques même dans le pire des cas, assurant ainsi une efficacité constante.

Ce qu’il faut retenir & Références

- **Synthèse :**
  - Les **arbres binaires de recherche (ABR)** sont fondamentaux pour manipuler des ensembles de données dynamiques et ordonnés.
  - L’intégration de mécanismes d’**équilibrage automatique** (comme dans les arbres AVL ou Rouge-Noir) est essentielle.
  - Ces mécanismes garantissent une performance optimale et stable (O(log n)) même dans les scénarios les plus défavorables, grâce à des règles spécifiques et des opérations de rotation.
- **Sources utilisées :**
  - Binary Search Tree - GeeksforGeeks
  - AVL Tree - Programiz
  - Red Black Tree - GeeksforGeeks
  - [Introduction to Algorithms (Cormen et al.) - Chapitre sur arbres équilibrés]
  - Wikipedia - Binary Search Tree
  - Wikipedia - AVL Tree